Microservices Design Patterns — Study Notes

Personal study notes taken while working through *Grokking Microservices Design Patterns* (Design Gurus, Arslan Ahmad). These are my own condensed summaries — not a copy of the course. All 93 lessons across 14 chapters were read; the 12 patterns are written up below.

- **Level:** Intermediate · **93 lessons** · 14 chapters
- **Captured:** 2026-07-27

> Microservices design patterns are architectural solutions tailored to the microservices style. Each pattern is a template for solving one problem that arises when you build software as a suite of small, independently deployable services.

The twelve patterns

#	Pattern	What it solves	Notes
2	**Strangler Fig**	Gradually replace a monolith without a big-bang rewrite	02-strangler-fig.md
3	**API Gateway**	Single entry point; clients stop knowing your service topology	03-api-gateway.md
4	**Backends for Frontends**	One tailored backend per client type; stop over-fetching	04-bff.md
5	**Service Discovery**	Find services by name when IPs change constantly	05-service-discovery.md
6	**Circuit Breaker**	Stop calling a failing dependency; prevent cascading failure	06-circuit-breaker.md
7	**Bulkhead**	Partition resources so one failure can't starve everything	07-bulkhead.md
8	**Retry**	Ride out transient faults with backoff and jitter	08-retry.md
9	**Sidecar**	Cross-cutting concerns as a co-located helper process	09-sidecar.md
10	**Saga**	Distributed transactions via local steps + compensation	10-saga.md
11	**Event-Driven Architecture**	Loose coupling and real-time reaction via events	11-event-driven.md
12	**CQRS**	Separate read and write models so each can be optimised	12-cqrs.md
13	**Configuration Externalization**	Change config without redeploying	13-config-externalization.md

---

Quick reference — grouped by concern

Migration

Pattern	One-line
Strangler Fig	Facade routes traffic to old or new system; migrate capability by capability, then delete the legacy system

Edge / routing

Pattern	One-line
API Gateway	One entry point handling routing, aggregation, auth, rate limiting, caching, logging
BFF	One gateway *per client type*, each shaping responses for that client's needs

Communication

Pattern	One-line
Service Discovery	Registry of live instances; look up by logical name. Client-side (Eureka+Ribbon) vs. server-side (Kubernetes DNS)

Resilience — the trio that works together

Pattern	One-line	Bounds…
Circuit Breaker	Closed → Open → Half-Open; fail fast once a dependency is clearly down	*how long* you keep trying
Bulkhead	Separate thread/resource pools per dependency	*how much* damage one failure can do
Retry	Re-attempt transient failures with exponential backoff + jitter	*how many* attempts before giving up

Cross-cutting concerns

Pattern	One-line
Sidecar	Co-located helper process (Envoy/Istio) absorbing networking, security, observability

Data consistency & messaging

Pattern	One-line
Saga	Sequence of local transactions, each with a compensating transaction. Choreography (events) vs. orchestration (coordinator)
Event-Driven	Producers publish events to a bus; consumers react. Broker / Mediator / Hybrid
CQRS	Commands change state and return nothing; queries return data and change nothing

Operations

Pattern	One-line
Config Externalization	Central config server (e.g. Spring Cloud Config over Git); fetch at runtime, refresh without restart

---

Cross-cutting themes worth noticing

These recur across nearly every pattern — they're the real content of the course:

1. **Everything centralized becomes a single point of failure.** API Gateway, service registry, config server, saga orchestrator, mediator. The answer is always the same: run it HA, and **cache last-known-good on the client** so an outage degrades rather than kills.

2. **Every indirection costs a network hop.** Gateway, BFF, sidecar, service discovery. Individually negligible, cumulatively not — and each hop is a new failure mode.

3. **Idempotency is non-negotiable** once anything can be retried — which in a distributed system is everything. Shows up in Retry, Saga, and Event-Driven.

4. **Eventual consistency is the price of independence.** Saga, CQRS with separate databases, and EDA all trade immediate consistency for autonomy and availability. Decide whether the business can accept the window.

5. **The patterns compose.** Circuit Breaker + Bulkhead + Retry is one resilience story. Saga + EDA + CQRS is one data story. Gateway/BFF + Discovery + Sidecar is one traffic story. Interview answers land better when you name the neighbours.

6. **Complexity is the universal cost.** Every single chapter's "considerations" section says some version of: harder to debug, harder to test, more moving parts, steeper learning curve. The pattern is only worth it when the problem is real.

> "A good architecture is not one that uses every pattern known to mankind but one that elegantly solves the problem at hand." — **Grady Booch**

Note format used in each file

1. **Problem** — what breaks without it 2. **Solution** — the core idea 3. **Architecture** — components and flow 4. **Worked example** — the code, with notes on what it simplifies 5. **Trade-offs & performance** — costs, latency, failure modes 6. **When to use / when to avoid** 7. **Real-world system design examples** 8. **Interview soundbite** — the 30-second version 9. **Related patterns** — cross-links